Comparison between Wagner's and Theodorsen's Representation of Flutter Predicition for Thin Plated Airfoils

Stach Mathias Andrzej Vleeshouwers

November 9, 2026

Contents

1 Method **2**
1.1 Assumptions and Aerodynamic Model 2
1.2 Theodorsen's representation 3
1.3 Wagners Representation 4

2 Results **7**
2.1 P-K Method to solve Theodorsen's representation 7
2.1.1 Results 7
2.2 Wagner's Method 7
2.2.1 Results 7

3 Discussion **8**
3.1 Comparison 8

4 Conclusion **9**

5 Decleration of Honesty **9**

6 Appendix **10**
6.1 P-K Method Matlab Code 10
6.2 Wagner's Method Matlab Code 13
6.3 Plotting Code 16

1 Method

1.1 Assumptions and Aerodynamic Model

For the unsteady analysis of flutter on an airfoil the following assumptions were made: an unsteady 2-Dimensional incompressible and irrotational flow. The two degrees of freedom, pitch and heave, are assumed to stay close to their starting locations with no flow separation occurring. Finally the Kutta Condition and Kelvins theorem are assumed to hold. The model for the plunge pitch airfoil can be seen in Figure 1. The airfoil is modelled as a thin plate where the flow tangency condition boundary condition is modelled as found in Equation 1, when S_u and S_l are the upper and lower surfaces respectively.

$$\frac{\partial \phi}{\partial n} = w(x,t) \quad x \subset S_u, \frac{\partial \phi}{\partial n} = -w(x,t) \quad x \subset S_l \tag{1}$$

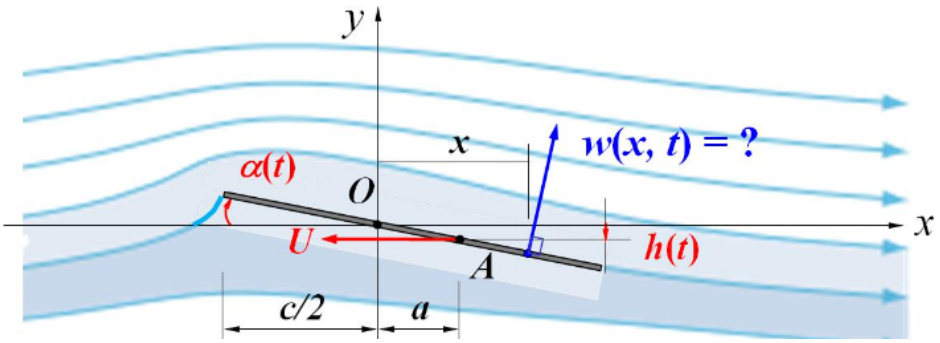


Figure 1: Plunge Pitch model of thin plate Airfoil

Through these assumptions it can be assumed that the flow is given by a potential function $\phi(x,y)$. For the potential the velocity component in x-direction u is given by $\frac{\partial \phi}{\partial x} = u$ and the component in y direction v is given by $\frac{\partial \phi}{\partial y} = v$. From the flow tangency boudary condition the potential flow around the flat plate is modeled as sheet of sources on the upper side of the wing and a sheet of sinks on the lower side. These source sheets are functions of both space and time. Additionally the wake vortexes created by changing pitch and heave create a vortex potential line off the trailing edge of the airfoil, which in turn induce pressure differences on the airfoil.

The nonhomogeneus equations of motion of the flat plate representation can be found in Equation 2.

$$\begin{bmatrix} m & -m \cdot a \\ -m \cdot a & I_\alpha \end{bmatrix} \begin{Bmatrix} \ddot{h} \\ \ddot{\alpha} \end{Bmatrix} + \begin{bmatrix} K_h & 0 \\ 0 & K_\alpha \end{bmatrix} \begin{Bmatrix} h \\ \alpha \end{Bmatrix} = \begin{Bmatrix} -L \\ M \end{Bmatrix} \tag{2}$$

The parameters used for the flutter analysis in this report can be found in Table 1.

Table 1: Structural and aerodynamic parameters

Material Density	$\mu = 1000$ kg/m^3
Chord	$c = 2$ m
Location of elastic axis from leading-edge	$x_f = 0.48c \quad (a = -0.02c)$
Thickness	$t = 0.01$ m
Length	$l = 1.0$ m
Plunge natural frequency	$\omega_h = 4$ Hz
Pitch natural frequency	$\omega_\alpha = 10$ Hz
Plunge stiffness	$K_h = m\omega_h^2 = 1.2633\text{e+}04$
Pitch stiffness	$K_\alpha = I_\alpha \omega_\alpha^2 = 2.6319\text{e+}04$
Air density	$\rho = 1.225$ kg/m^3

1.2 Theodorsen's representation

Theodorsen assumes that the motion of the airfoil is entirely harmonic. The Theodorsen function $C(k)$ describes the relationship between the circulatory lift response and the corresponding steady-state lift under harmonic motion. Through derivation that is outside the scope of this report, the unsteady lift force acting on the flat plate can be found in Equation 4 and the moment acting on the flat plate can be found in Equation 5. It is to be noted that both the lift and moment both depend on the current state as well as velocity and acceleration components of the heave and pitch states. An approximate version of the Theodorsen function can be seen in Equation 3. Note the the Theodorsen function is a complex valued function.

$$C(k) = 1 - \frac{0.165}{1 - i \cdot 0.0455/k} - \frac{0.335}{1 - i \cdot 0.3/k} \tag{3}$$

$$L = \rho\pi b^2 \left[U\dot{\alpha} - ab\ddot{\alpha} + \ddot{h}\right] + 2\pi\rho bUC(k) \left[U\alpha + \dot{h} - \left(a - \frac{b}{2}\right)\dot{\alpha}\right] \tag{4}$$

$$M = \rho\pi b^2 \left[a\ddot{h} + U\dot{h} + U^2\alpha - \left(a^2 + \frac{b^2}{8}\right)\ddot{\alpha}\right] - \rho\pi bU\left[b - (2a+b)C(k)\right] \left[U\alpha + \dot{h} - \left(a - \frac{b}{2}\right)\dot{\alpha}\right] \tag{5}$$

Inserting the derived equations for L, M the full homogeneus equations can be found in Equation 6

$$\begin{aligned}
&\begin{bmatrix} m & -ma \\ -ma & I_\alpha \end{bmatrix} \begin{Bmatrix} \ddot{h} \\ \ddot{\alpha} \end{Bmatrix} + \begin{bmatrix} K_h & 0 \\ 0 & K_\alpha \end{bmatrix} \begin{Bmatrix} h \\ \alpha \end{Bmatrix} \\
&= \begin{bmatrix} -\rho\pi b^2 & a\rho\pi b^2 \\ a\rho\pi b^2 & -\left(a^2 + \frac{b^2}{8}\right)\rho\pi b^2 \end{bmatrix} \begin{Bmatrix} \ddot{h} \\ \ddot{\alpha} \end{Bmatrix} \\
&\quad + \begin{bmatrix} -2\pi\rho bUC(k) & -\rho\pi b^2 U + \left(a - \frac{b}{2}\right)2\pi\rho bUC(k) \\ \rho\pi b^2 U - \rho\pi bU\left(b - (2a+b)C(k)\right) & \rho\pi bU\left(b - (2a+b)C(k)\right)\left(a - \frac{b}{2}\right) \end{bmatrix} \begin{Bmatrix} \dot{h} \\ \dot{\alpha} \end{Bmatrix} \\
&\quad + \begin{bmatrix} 0 & -2\pi\rho bU^2 C(k) \\ 0 & \rho\pi b^2 U^2 - \rho\pi bU^2\left(b - (2a+b)C(k)\right) \end{bmatrix} \begin{Bmatrix} h \\ \alpha \end{Bmatrix}
\end{aligned} \tag{6}$$

To simplify the degrees of freedom and to create a state space model of the system new

coordinates are introduced which can be found in Equation 7.

$$\{q\} = \begin{Bmatrix} h \\ \alpha \end{Bmatrix}, \qquad \{z\} = \begin{Bmatrix} q \\ \dot{q} \end{Bmatrix} = \begin{Bmatrix} h \\ \alpha \\ \dot{h} \\ \dot{\alpha} \end{Bmatrix} \tag{7}$$

The compact form of the equations of motion can be found in Equation 8 with the newly defined coordinates.

$$[M]\{\ddot{q}\} + [K]\{q\} = [D]\{\ddot{q}\} + [E(k)]\{\dot{q}\} + [F(k)]\{q\} \tag{8}$$

The state space model is then given by Equation 9 where A is given in Equation 10.

$$\{\dot{z}\} = [A(k)]\{z\} \tag{9}$$

$$[A(k)] = \begin{bmatrix} [0] & [I] \\ [M-D]^{-1}[F(k)-K] & [M-D]^{-1}[E(k)] \end{bmatrix} \tag{10}$$

From the matrix A the stability of the system can be determined by checking the location of its eigenvalues. Since the matrixes E and F both depend on the reduced frequency k for the Wagners function. k is defined in Equation 11.

$$k = \frac{\omega \cdot b}{U} \tag{11}$$

Flutter occurs when the system starts to become unstable. This occurs when the eigenvalues of the matrix enter the right half of the complex plane; ie. the real part of the eigenvalue becomes positive. So the flutter condition is $Re(\lambda) = 0$, which is the transition point. Once the flutter condition is passed, for any eigenvalue, exponentially growing harmonic motion occurs, which is flutter. The airspeed where the first eigenvalue fulfills the flutter condition is the flutter speed. The flutter frequency is then $\omega_f = |\lambda| = |Im(\lambda)|$. However since the Wagners function depends on reduced frequency k, which itself depends on the airspeed as well as the flutter frequency, an iterative procedure would be required to find a solution.

1.3 Wagners Representation

Wagner's representation describes the aerodynamic response to changes in effective angle of attack over time. Instead of directly using the frequency-dependent Theodorsen function, we introduce aerodynamic lag states to account for the wake's memory. The Wagner function ($\phi(U_\infty t/b)$) describes the change in lift when a change in the effective angle of attack in introduced. It assumes an instantaneous step change in angle of attack, which changes the magnitude of lift force acting on the airfoil. It assumes that the increase in instantaneous lift at the start of motion is equal to half the value of the increase in lift that it would have at steady state conditions, at that angle of attack. This introduces aerodynamic lag into the aeroelastic model. An approximate form of the Wagner function can be found in Equation 12.

$$\Phi(t) = 1 - \Psi_1 e^{-\varepsilon_1 U_\infty t/b} - \Psi_2 e^{-\varepsilon_2 U_\infty t/b} \tag{12}$$

$$\Psi_1 = 0.165, \qquad \Psi_2 = 0.335, \qquad \varepsilon_1 = 0.0455, \qquad \varepsilon_2 = 0.30$$

Through derivation that is outside the scope of this report the lift and moment acting on the airfoil can be found in Equation 13 and Equation 14 respectively. $w_1(t), w_2(t), w_3(t), w_4(t)$ are auxiliary aerodynamic lag states that represent the history-dependent contributions of plunge and pitch motion to the unsteady aerodynamic lift. w_1, w_2 account for the history of plunge

$h(t)$ and w_3, w_4 account for the history of pitch $\alpha(t)$. The governing differential equations for $w_1(t), w_2(t), w_3(t), w_4(t)$ can be seen in Equation 15.

$$\begin{aligned}\frac{l(t)}{\rho\pi U c} = & \left[h(0) + \left(\frac{3c}{4} - x_f\right)\alpha(0)\right]\dot{\Phi}(t) \\ & + \left[U\alpha(t) + \dot{h}(t) + \left(\frac{3c}{4} - x_f\right)\dot{\alpha}(t)\right]\Phi(0) \\ & - \left[h(t) + \left(\frac{3c}{4} - x_f\right)\alpha(t)\right]\dot{\Phi}(0) \\ & - \Psi_1\left(\frac{\varepsilon_1 U}{b}\right)^2 w_1(t) - \Psi_2\left(\frac{\varepsilon_2 U}{b}\right)^2 w_2(t) \\ & + \Psi_1\frac{\varepsilon_1 U^2}{b}\left[1 - \varepsilon_1(1-2e)\right]w_3(t) \\ & + \Psi_2\frac{\varepsilon_2 U^2}{b}\left[1 - \varepsilon_2(1-2e)\right]w_4(t)\end{aligned} \tag{13}$$

$$m(t) = ec\,l(t) \tag{14}$$

$$\begin{aligned}\dot{w}_1 &= h - \frac{\varepsilon_1 U_\infty}{b}w_1 \\ \dot{w}_2 &= h - \frac{\varepsilon_2 U_\infty}{b}w_2 \\ \dot{w}_3 &= \alpha - \frac{\varepsilon_1 U_\infty}{b}w_3 \\ \dot{w}_4 &= \alpha - \frac{\varepsilon_2 U_\infty}{b}w_4\end{aligned} \tag{15}$$

Now that the forces and moments are defined we can input them into the equations of motion in Equation 2. The final equations of motion can be found in Equation 16. The Matrixes M, C and K are defined below. This is a nonhomegeneus system of differential equations. However if the initial conditions are set to zero, the differential equation becomes homogeneous as the forcing function becomes zero.

$$[M]\begin{Bmatrix}\ddot{h}\\ \ddot{\alpha}\end{Bmatrix} + [C]\begin{Bmatrix}\dot{h}\\ \dot{\alpha}\end{Bmatrix} + [K]\begin{Bmatrix}h\\ \alpha\end{Bmatrix} + [W]\begin{Bmatrix}w_1\\ w_2\\ w_3\\ w_4\end{Bmatrix} = \begin{Bmatrix}\rho\pi U c\dot{\Phi}(t)\left[h(0) + \left(\frac{3c}{4} - x_f\right)\alpha(0)\right] \\ -\rho\pi U e c^2\dot{\Phi}(t)\left[h(0) + \left(\frac{3c}{4} - x_f\right)\alpha(0)\right]\end{Bmatrix} \tag{16}$$

$$[M] = \begin{bmatrix} m + \rho\pi b^2 & S - \rho\pi b^2\left(x_f - \frac{c}{2}\right) \\ S - \rho\pi b^2\left(x_f - \frac{c}{2}\right) & I_\alpha + \rho\pi b^2\left[\left(x_f - \frac{c}{2}\right)^2 + \frac{b^2}{8}\right]\end{bmatrix}$$

$$[C] = \rho\pi U c\begin{bmatrix} \Phi(0) & \frac{c}{4} + \Phi(0)\left(\frac{3c}{4} - x_f\right) \\ -ec\Phi(0) & \left(\frac{3c}{4} - x_f\right)\left[\frac{c}{4} - ec\Phi(0)\right]\end{bmatrix}$$

$$[K] = \begin{bmatrix} K_h + \rho\pi U c\dot{\Phi}(0) & \rho\pi U c\left[U\Phi(0) + \left(\frac{3c}{4} - x_f\right)\dot{\Phi}(0)\right] \\ -\rho\pi U e c^2\dot{\Phi}(0) & K_\alpha - \rho\pi U e c^2\left[U\Phi(0) + \left(\frac{3c}{4} - x_f\right)\dot{\Phi}(0)\right]\end{bmatrix}$$

$$[W] = 2\rho\pi U^3\begin{bmatrix} -\frac{\Psi_1\varepsilon_1^2}{b} & -\frac{\Psi_2\varepsilon_2^2}{b} & \Psi_1\varepsilon_1[1-\varepsilon_1(1-2e)] & \Psi_2\varepsilon_2[1-\varepsilon_2(1-2e)] \\ \frac{ec\Psi_1\varepsilon_1^2}{b} & \frac{ec\Psi_2\varepsilon_2^2}{b} & -ec\Psi_1\varepsilon_1[1-\varepsilon_1(1-2e)] & -ec\Psi_2\varepsilon_2[1-\varepsilon_2(1-2e)]\end{bmatrix}$$

Finally to complete the analysis the equations of motion defining the system will be brought into a state space representation. 8 degrees of freedom are required. The state space vector and state space equation can be found in Equation 17.

$$\{z\} = \begin{Bmatrix} h \\ \alpha \\ \dot{h} \\ \dot{\alpha} \\ w_1 \\ w_2 \\ w_3 \\ w_4 \end{Bmatrix}, \qquad \{\dot{z}\} = [A(U)]\{z\} + [D] \tag{17}$$

Through manipulation of Equation 16 the matrixes A and D can be found, as seen in Equation 18. RHS is the right hand side and nonhomogeneus part of Equation 16. The B and G matrixes are the matrixes defining the differential equation governing $w_1(t), w_2(t), w_3(t), w_4(t)$.

$$[A] = \begin{bmatrix} -[M]^{-1}[C] & -[M]^{-1}[K] & -[M]^{-1}[W] \\ I & 0 & 0 \\ 0 & B & G \end{bmatrix}, [D] = \begin{bmatrix} [M]^{-1}\{RHS\} \\ 0 \\ 0 \end{bmatrix} \tag{18}$$

$$B = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}, \qquad G = \begin{bmatrix} -\frac{\varepsilon_1 U}{b} & 0 & 0 & 0 \\ 0 & -\frac{\varepsilon_2 U}{b} & 0 & 0 \\ 0 & 0 & -\frac{\varepsilon_1 U}{b} & 0 \\ 0 & 0 & 0 & -\frac{\varepsilon_2 U}{b} \end{bmatrix}$$

The A matrix is a 8x8 matrix representing the system dynamics of the airfoil. However contrary to the Theodorsen representation the A matrix is only dependant on the airspeed and no frequency components, thus no iterative solution method is required to converge to a solution. For a certain airspeed the flutter condition $Re(\lambda) = 0$ will be obtained and a eigenvalue of the A matrix will become complex. The airspeed at which this occurs is the flutter speed. At the flutter condition the flutter frequency can be determined $\omega_f = |\lambda| = |Im(\lambda)|$.

2 Results

2.1 P-K Method to solve Theodorsen's representation

This section will present the solutions and results to the frequency domain aeroelastic modelling presented in subsection 1.2. Since the matrixes describing the system depend on both the flutter frequency and velocity an iterative approach is required to solve the problem. For this analysis the P-K method was chosen. This analysis checked to flutter condition for all airspeeds up to 100$[m/s]$. The exact iteration scheme can be found in subsection 6.1.

2.1.1 Results

The flutter speed and frequency were determined at the point where the first eigenvalue of either the plunge or pitch mode crossed into the right half of the complex plane. In this analysis this occured for the pitch mode at an airspeed U_f of 82.94 $[m/s]$. The pitch frequency ω_f at this airspeed was found to be 7.252 [Hz]. As shown in Figure 2, the change in the eigenmodes and frequencies can be seen. The corresponding modal frequencies are shown in Figure 3.

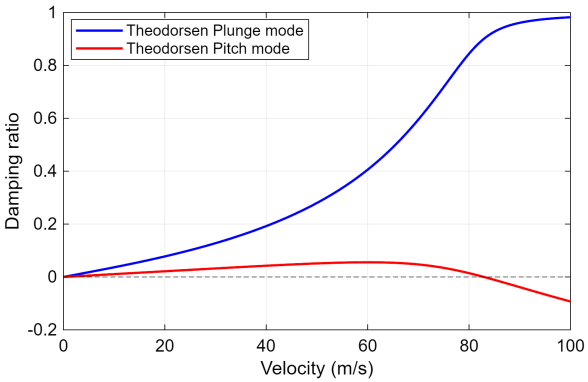


Figure 2: Damping ratio against airspeed (Theodorsen)

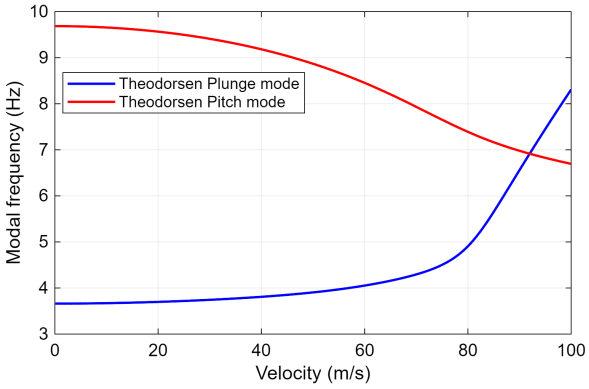


Figure 3: Modal frequency against airspeed (Theodorsen)

2.2 Wagner's Method

This section will present the results found in using Wagner's time domain representation of the aeroelastic flutter modelling as presented in subsection 1.3. As Wagner's method only requires airspeed to form the matrices, the eigenfrequencies and modes can be found at each airspeed, without iteration. The solution method can be found in subsection 6.3.

2.2.1 Results

The flutter airspeed U_f was determined to b 82.95/$[m/s]$, which is when the pitch mode crossed into the right half complex plane. The corresponding eigenfrequency for the pitch mode was 7.253 [Hz]. The corresponding damping ratio and eigenfrequency plots for each airspeed can be seen in Figure 4 and Figure 5 respectively.

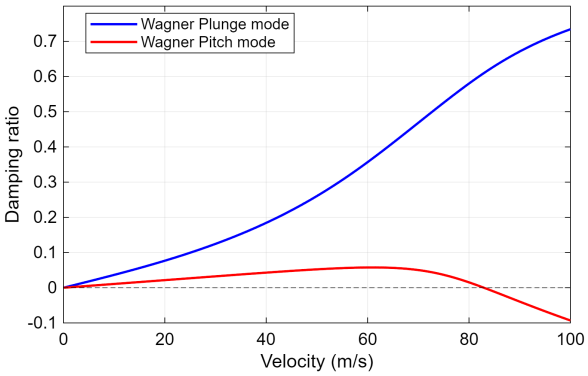


Figure 4: Damping ratio against airspeed (Wagner)

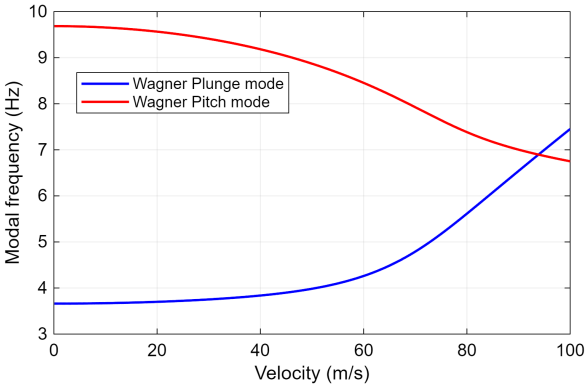


Figure 5: Modal frequency against airspeed (Wagner)

3 Discussion

In the following section the results from the two solution methods will be discussed.

3.1 Comparison

The results as illustrated in section 2 for the damping ratio and modal frequency can be seen in Figure 6 and Figure 7. At low airspeeds, the behaviour of both models is very similar in

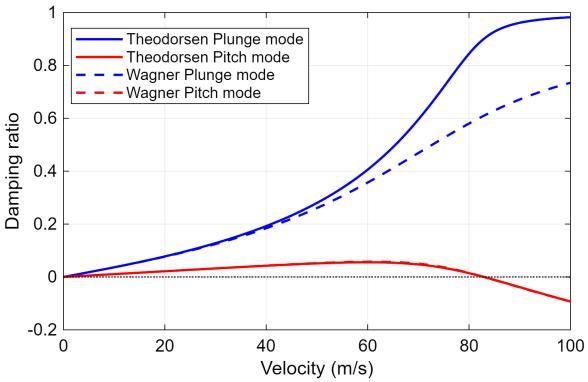


Figure 6: Comparison of plunge and pitch modal damping ratios obtained using the Theodorsen and Wagner models.

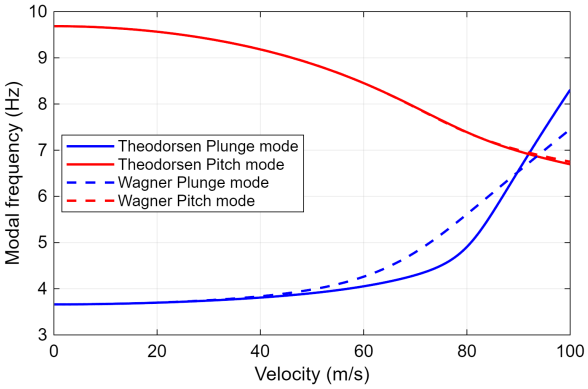


Figure 7: Comparison of plunge and pitch modal frequencies obtained using the Theodorsen and Wagner models.

terms of both pitch and heave. The pitch and heave responses both start at a modal frequency that is slightly below the natural structural frequency. This is due to the 'added mass' of the surrounding air which decreases its natural frequency. As the airspeed transitions towards the flutter point, however, the plunge eigenvalues begin to differ in magnitude and angle. The pitch eigenvalues remain extremely similar until the final airspeed. Both models converge on a very similar flutter speed. The P-K method is most accurate at flutter because there is purely harmonic motion without any damping or exponential growth, as the model assumes. The PK method is entirely based on a harmonic motion assumption, so outside the flutter point, it is merely an approximation. As airspeed and damping ratio increase beyond the flutter point, the mode approaches non-oscillatory behaviour, rendering the harmonic assumption increasingly inaccurate and causing the PK result to lose accuracy. The Wagner method does not run into this problem since it doesn't make any assumptions of the motion of the airfoil, resulting in higher accuracy at higher damping ratios. However this comes at a higher computational cost since

ti involves adding four additional aerodynamic states and solving more eigenvalue problems for larger matrices.
Both models produce very similar results in determining flutter modes, which is logical given that they stem from the same aerodynamic principles and assumptions.

4 Conclusion

This aeroelastic analysis of a flat plat wing model is good for an intial first order estimate of a wings flutter point. However to improve the accuracy of the flutter predicition more accurate methods would have to be used. Primarily 3D-Wing effects would have to be taken into account, as well as compressibility effects, since most airplanes fly at high mach numbers, where compressibility effects have to be taken into account. Furthermore most aerofoils are not flat plates but have camber and thickness distributions, which changes the lift and moment dynamics as it moves through different angles of attack.

5 Decleration of Honesty

During the preparation of this work, the author used generative AI tools (ChatGPT, Claude) for the following purposes: • Generating and Formatting Figures in Matlab and constructing Matrices • Writing Latex tables and matrices AI tools were not used to perform the underlying physical analysis or formulate the conclusions of this study. After using these tools, the author reviewed and edited all output and take full responsibility for the content of this report.

6 Appendix

6.1 P-K Method Matlab Code

```
clear; clc; close all


mew=1000; %kg/m^3
rho=1.225; %air density
c=2; %chord in meters
x_f= 0.48*c;
a=-0.02*c;
t=0.01; %thickness in meters
l=1; %length in meters
omega_h=4; %plunge natural frequency in HZ
omega_a=10; %pitch natural frequency in HZ
b=c/2; %half of the chord length

m=c*mew*t*l;
I_a=1/12*m*c^2+m*a^2;
k_plunge= m*(2*pi*omega_h)^2;
k_pitch= I_a*(2*pi*omega_a)^2; %converting to rad/s
%PK method
%loop parameters
du=0.01; %m/s
error_margin=1e-7;
Umin=0.1;
Umax=100; %m/s
%matrixes
K=[k_plunge 0; 0 k_pitch;];
M=[m -m*a; -m*a I_a;];
D=[-rho*pi*b^2 a*rho*pi*b^2; rho*pi*b^2*a -(a^2+b^2/8)*rho*pi*b^2;];
W1 = []; d1 = []; W2 = []; d2 = []; d_all = [];
Urange = Umin:du:Umax;


for U= Urange
    ii=3;
    for om=[sqrt(k_plunge/m) sqrt(k_pitch/I_a)]
    err =10;
    iter=0;
        while err >=error_margin
          iter=iter+1;
          k=om*b/U;
          C=1-0.165/(1-(0.0455*1i/k))-(0.335/(1-1i*(0.3/k)));
          E=[-2*pi*rho*b*U*C -rho*pi*b^2*U+(a-b/2)*2*pi*rho*b*U*C; rho*
              pi*b^2*U-rho*pi*b*U*(b-(2*a+b)*C) rho*pi*b*U*(b-(2*a+b)*C)
              *(a-b/2);];
          F=[0 -2*pi*rho*b*U^2*C; 0 rho*pi*b^2*U^2-rho*pi*b*U^2*(b-(2*a+
              b)*C)];
          A=[zeros(2) eye(2);(M-D)\(F-K) (M-D)\E;];
          d=eig(A);
          [d_imag, index] = sort(imag(d), 'ascend'); %sort eigenvalues
              in ascending order
          d_imag1 = d_imag(ii); % if heave/pitch, then extract the
              imaginary of second largest/largest
          err = abs(d_imag1-om); % Calculate error of computed imaginary
               (d_imag1) against the assumed freq (om) in the first step
```

```
49            om = d_imag1;
50          end
51      d_sorted=d(index);
52      %W stores natural frequencies
53      %d stores damping ratios
54      %d_all stores all eigenvalues of all airspeeds.
55      if ii==3;W1=[W1 abs(d_sorted(ii))]; d1 = [d1 -real(d_sorted(ii))/abs
            (d_sorted(ii))]; end
56      if ii == 4; W2 = [W2 abs(d_sorted(ii))]; d2 = [d2 -real(d_sorted(ii)
            )/abs(d_sorted(ii))]; end
57      d_all=[d_all d_sorted]; %this stores all of the eigenvalues for the
            two modes
58      ii=ii+1;
59      end
60  end
61  %need to find where the first eigenvalue crosses into the right half
        plane
62  %ie. where the damping ratio becomes negative.
63  ind=1;
64  running=true;
65  double mode;
66  while running
67      if d1(1,ind)<=0; running=false; mode=1; break; end
68      if d2(1,ind)<=0; running=false; mode=2; break; end
69      ind=ind+1;
70  end
71  Uflutter=Urange(1,ind)
72  flutterfrequency=0;
73  if mode==1;flutterfrequency=W1(ind);end
74  if mode==2;flutterfrequency=W2(ind);end
75  frequency=flutterfrequency/2/pi %converting to HZ
76
77
78
79  % Figure 1: Damping ratio
80  fig1 = figure('Color', 'white');
81  ax1 = axes(fig1);
82
83  plot(ax1, Urange, d1, 'b-', Urange, d2, 'r-', 'LineWidth', 1.5);
84  yline(ax1, 0, 'k--');
85
86  xlabel(ax1, 'Velocity␣(m/s)', 'Color', 'black');
87  ylabel(ax1, 'Damping␣ratio', 'Color', 'black');
88
89  lgd1 = legend(ax1, 'Theodorsen␣Plunge␣mode', ...
90      'Theodorsen␣Pitch␣mode', 'Location', 'best');
91  lgd1.Color = 'white';
92  lgd1.EdgeColor = 'black';
93  lgd1.TextColor = 'black';
94
95  grid(ax1, 'on');
96  ax1.Color = 'white';
97  ax1.XColor = 'black';
98  ax1.YColor = 'black';
99  ax1.GridColor = [0.7 0.7 0.7];
100 ax1.FontSize = 11;
101 ax1.Box = 'on';
102
```

```
103
104 % Figure 2: Modal frequency
105 fig2 = figure('Color', 'white');
106 ax2 = axes(fig2);
107
108 plot(ax2, Urange, W1/2/pi, 'b-', Urange, W2/2/pi, 'r-', 'LineWidth',
    1.5);
109
110 xlabel(ax2, 'Velocity (m/s)', 'Color', 'black');
111 ylabel(ax2, 'Modal frequency (Hz)', 'Color', 'black');
112
113 lgd2 = legend(ax2, 'Theodorsen Plunge mode', ...
114     'Theodorsen Pitch mode', 'Location', 'best');
115 lgd2.Color = 'white';
116 lgd2.EdgeColor = 'black';
117 lgd2.TextColor = 'black';
118
119 grid(ax2, 'on');
120 ax2.Color = 'white';
121 ax2.XColor = 'black';
122 ax2.YColor = 'black';
123 ax2.GridColor = [0.7 0.7 0.7];
124 ax2.FontSize = 11;
125 ax2.Box = 'on';
126
127
128 % Export figures
129 exportgraphics(fig1, 'figure1.png', ...
130     'BackgroundColor', 'white', 'Resolution', 300);
131
132 exportgraphics(fig2, 'figure2.png', ...
133     'BackgroundColor', 'white', 'Resolution', 300);
```

Listing 1: P-K Method Flutter Analysis

6.2 Wagner's Method Matlab Code

```
clear; clc; close all
global b Psi1 Psi2 epsilon1 epsilon2

mew=1000; %kg/m^3
rho=1.225; %air density
c=2; %chord in meters
xf= 0.48*c;
a=-0.02*c;
t=0.01; %thickness in meters
l=1; %length in meters
omega_h=4; %plunge natural frequency in HZ
omega_a=10; %pitch natural frequency in HZ
b=c/2; %half of the chord length
e=(xf-c/4)/c;

m=c*mew*t*l;
I_alpha=1/12*m*c^2+m*a^2;
k_plunge= m*(2*pi*omega_h)^2;
k_pitch= I_alpha*(2*pi*omega_a)^2; %converting to rad/s
S=-m*a;
Psi1=0.165;
Psi2=0.335;
epsilon1=0.0445;
epsilon2=0.3;
function [phi]= phi(t, U)
 global b Psi1 Psi2 epsilon1 epsilon2
 x=U*t/b;
 phi = 1-Psi1*exp(-x*epsilon1)-Psi2*exp(-x*epsilon2);
end
function Phi_dot = Phi_dot(t, U)
 global b Psi1 Psi2 epsilon1 epsilon2
    Phi_dot = (U/b) * ( ...
        Psi1*epsilon1*exp(-epsilon1*U*t/b) + ...
        Psi2*epsilon2*exp(-epsilon2*U*t/b));
end

%assuming starting from rest so IC=0


%loop parameters
du=0.01; %m/s
error_margin=1e-7;
Umin=0.1;
Umax=100; %m/s

W1_W = []; d1_W = []; W2_W = []; d2_W= [];
Urange = Umin:du:Umax ;
for U = Urange
t=0;
Phi0=phi(0,U);
Phi_dot0=Phi_dot(0, U);

% Mass matrix
M = [
    m + rho*pi*b^2, ...
    S - rho*pi*b^2*(xf - c/2);
```

```
    S - rho*pi*b^2*(xf - c/2), ...
    I_alpha + rho*pi*b^2*((xf - c/2)^2 + b^2/8)
];

% Damping matrix
C = rho*pi*U*c * [
    Phi0, ...
    c/4 + Phi0*(3*c/4 - xf);

    -e*c*Phi0, ...
    (3*c/4 - xf)*(c/4 - e*c*Phi0)
];

% Stiffness matrix
K = [
    k_plunge + rho*pi*U*c*Phi_dot0, ...
    rho*pi*U*c*(U*Phi0 + (3*c/4 - xf)*Phi_dot0);

    -rho*pi*U*e*c^2*Phi_dot0, ...
    k_pitch - rho*pi*U*e*c^2*(U*Phi0 + (3*c/4 - xf)*Phi_dot0)
];

% Wagner lag-state coupling matrix
W = 2*rho*pi*U^3 * [
    -Psi1*epsilon1^2/b, ...
    -Psi2*epsilon2^2/b, ...
    Psi1*epsilon1*(1 - epsilon1*(1 - 2*e)), ...
    Psi2*epsilon2*(1 - epsilon2*(1 - 2*e));

    e*c*Psi1*epsilon1^2/b, ...
    e*c*Psi2*epsilon2^2/b, ...
    -e*c*Psi1*epsilon1*(1 - epsilon1*(1 - 2*e)), ...
    -e*c*Psi2*epsilon2*(1 - epsilon2*(1 - 2*e))
];

B=[1 0; 1 0; 0 1; 0 1;];
G=[-epsilon1*U/b 0 0 0; 0 -epsilon2*U/b 0 0; 0 0 -epsilon1*U/b 0; 0 0 0
    -epsilon2*U/b;];
Q = [
    -M\C, -M\K, -M\W;
    eye(2), zeros(2,2), zeros(2,4);
    zeros(4,2), B, G
];

D = eig(Q);
[D_imag,index] = sort(imag(D),'ascend');
D_sort = D(index,:);
W1_W = [W1_W, abs(D_sort(7))]; d1_W = [d1_W,-real(D_sort(7))/abs( D_sort
    (7))];
W2_W = [W2_W, abs(D_sort(8))]; d2_W = [d2_W,-real(D_sort(8))/abs( D_sort
    (8))];
end

%need to find where the first eigenvalue crosses into the right half
    plane
%ie. where the damping ratio becomes negative.
ind=1;
running=true;
```

```
double mode;
while running
    if d1_W(1,ind)<=0; running=false; mode=1; break; end
    if d2_W(1,ind)<=0; running=false; mode=2;break; end
    ind=ind+1;
end
Uflutter=Urange(1,ind)
flutterfrequency=0;
if mode==1;flutterfrequency=W1(ind);end
if mode==2;flutterfrequency=W2_W(ind);end
frequency=flutterfrequency/2/pi %converting to HZ

% Figure 1: Damping ratio
fig1 = figure('Color', 'white');
ax1 = axes(fig1);

plot(ax1, Urange, d1_W, 'b-', Urange, d2_W, 'r-', 'LineWidth', 1.5);
yline(ax1, 0, 'k--');

xlabel(ax1, 'Velocity (m/s)', 'Color', 'black');
ylabel(ax1, 'Damping ratio', 'Color', 'black');
lgd=legend(ax1, 'Wagner Plunge mode', 'Wagner Pitch mode', 'Location', 'best');
lgd.Color = 'white';
lgd.EdgeColor = 'black';  % Optional: black border
lgd.TextColor = 'black';
grid(ax1, 'on');
ax1.Color = 'white';
ax1.XColor = 'black';
ax1.YColor = 'black';
ax1.GridColor = [0.7 0.7 0.7];
ax1.FontSize = 11;
ax1.Box = 'on';


% Figure 2: Modal frequency
fig2 = figure('Color', 'white');
ax2 = axes(fig2);

plot(ax2, Urange, W1_W/2/pi, 'b-', Urange, W2_W/2/pi, 'r-', 'LineWidth', 1.5);

xlabel(ax2, 'Velocity (m/s)', 'Color', 'black');
ylabel(ax2, 'Modal frequency (Hz)', 'Color', 'black');
lgd=legend(ax2, 'Wagner Plunge mode', 'Wagner Pitch mode', 'Location', 'best');
lgd.Color = 'white';
lgd.EdgeColor = 'black';  % Optional: black border
lgd.TextColor = 'black';
grid(ax2, 'on');
ax2.Color = 'white';
ax2.XColor = 'black';
ax2.YColor = 'black';
ax2.GridColor = [0.7 0.7 0.7];
ax2.FontSize = 11;
ax2.Box = 'on';


```

```
% Export figures
exportgraphics(fig1, 'figure3.png', 'BackgroundColor', 'white', '
    Resolution', 300);
exportgraphics(fig2, 'figure4.png', 'BackgroundColor', 'white', '
    Resolution', 300);
```

Listing 2: P-K Method Flutter Analysis

6.3 Plotting Code

```
% have to run other two files first so that matrices are stored
    correctly.
% Figure 1: Damping ratio
fig1 = figure('Color', 'white');
ax1 = axes(fig1);
hold(ax1, 'on');

% Theodorsen
plot(ax1, Urange, d1, 'b-', 'LineWidth', 1.5);
plot(ax1, Urange, d2, 'r-', 'LineWidth', 1.5);

% Wagner
plot(ax1, Urange, d1_W, 'b--', 'LineWidth', 1.5);
plot(ax1, Urange, d2_W, 'r--', 'LineWidth', 1.5);

yline(ax1, 0, 'k:', 'LineWidth', 1);

xlabel(ax1, 'Velocity␣(m/s)', 'Color', 'black');
ylabel(ax1, 'Damping␣ratio', 'Color', 'black');

lgd1 = legend(ax1, ...
    'Theodorsen␣Plunge␣mode', ...
    'Theodorsen␣Pitch␣mode', ...
    'Wagner␣Plunge␣mode', ...
    'Wagner␣Pitch␣mode', ...
    'Location', 'best');

lgd1.Color = 'white';
lgd1.EdgeColor = 'black';
lgd1.TextColor = 'black';

grid(ax1, 'on');
ax1.Color = 'white';
ax1.XColor = 'black';
ax1.YColor = 'black';
ax1.GridColor = [0.7 0.7 0.7];
ax1.FontSize = 11;
ax1.Box = 'on';


% Figure 2: Modal frequency
fig2 = figure('Color', 'white');
ax2 = axes(fig2);
hold(ax2, 'on');

% Theodorsen
plot(ax2, Urange, W1/2/pi, 'b-', 'LineWidth', 1.5);
```

```
plot(ax2, Urange, W2/2/pi, 'r-', 'LineWidth', 1.5);

% Wagner
plot(ax2, Urange, W1_W/2/pi, 'b--', 'LineWidth', 1.5);
plot(ax2, Urange, W2_W/2/pi, 'r--', 'LineWidth', 1.5);

xlabel(ax2, 'Velocity (m/s)', 'Color', 'black');
ylabel(ax2, 'Modal frequency (Hz)', 'Color', 'black');

lgd2 = legend(ax2, ...
    'Theodorsen Plunge mode', ...
    'Theodorsen Pitch mode', ...
    'Wagner Plunge mode', ...
    'Wagner Pitch mode', ...
    'Location', 'best');

lgd2.Color = 'white';
lgd2.EdgeColor = 'black';
lgd2.TextColor = 'black';

grid(ax2, 'on');
ax2.Color = 'white';
ax2.XColor = 'black';
ax2.YColor = 'black';
ax2.GridColor = [0.7 0.7 0.7];
ax2.FontSize = 11;
ax2.Box = 'on';


% Export figures
exportgraphics(fig1, 'figure5.png', ...
    'BackgroundColor', 'white', 'Resolution', 300);

exportgraphics(fig2, 'figure6.png', ...
    'BackgroundColor', 'white', 'Resolution', 300);
```

Listing 3: P-K Method Flutter Analysis